Insertion Sort Pseudo Code

Lecture 15.pdf
PDF

read the pdf and write a pseudo code for insertion sort of integers from it

Based on the explanation and code snippet provided in the **Lecture 15 PDF**, here is a **pseudo code** for the **Insertion Sort algorithm** for sorting a list of integers:

🧠 Insertion Sort Pseudo Code (Integers)

```pgsql
procedure InsertionSort(array A, integer n)
    for i from 1 to n - 1 do
        current ← A[i]
        j ← i
        while j > 0 and A[j - 1] > current do
            A[j] ← A[j - 1]          // Shift the larger value to the right
            j ← j - 1
        end while
        A[j] ← current               // Insert the current value in the correct
position
    end for
end procedure
```

📌 Explanation

- Start with the second element (`i = 1`) because the first element is trivially "sorted".
- Store the current element in a temporary variable (`current`).
- Compare `current` with elements before it.
- While those elements are greater, shift them one position to the right.

- Finally, insert the `current` value into its correct sorted position.

This matches the behavior described in your PDF under the "Insertion sort algorithm" section and is consistent with the C++-style loop:

```cpp
for (int j = k; j > 0; --j) {
    if (array[j - 1] > array[j]) {
        std::swap(array[j - 1], array[j]);
    } else {
        break;
    }
}
```

Let me know if you'd like this converted into any specific programming language!

write the cpp code and then the pseudo code for bubble sort

Sure! Here's the **C++ code** for Bubble Sort, followed by the corresponding **pseudo code**.

✅ C++ Code for Bubble Sort (Simple Version)

```cpp
#include <iostream>
using namespace std;

void bubbleSort(int array[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (array[j] > array[j + 1]) {
                // Swap adjacent elements if they are in the wrong order
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
            }
        }
    }
}
```

```
int main() {
    int data[] = {64, 25, 12, 22, 11};
    int n = sizeof(data) / sizeof(data[0]);

    bubbleSort(data, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; ++i)
        cout << data[i] << " ";

    return 0;
}
```

🧠 Bubble Sort Pseudo Code

```
css

procedure BubbleSort(array A, integer n)
    for i from 0 to n - 2 do
        for j from 0 to n - i - 2 do
            if A[j] > A[j + 1] then
                swap A[j] and A[j + 1]
            end if
        end for
    end for
end procedure
```

Would you like the optimized version of Bubble Sort (with early exit using a flag)?

yes show me optimized version of Bubble Sort (with early exit using a flag) code and also the pseodu code for that. Explain in details what and how is improved in this optimized version

Great! Here's the **optimized version** of **Bubble Sort** that includes a **flag (Boolean variable)** to check whether any **swap** was made during a pass. If **no swap** occurred, the array is already

sorted, and the algorithm **terminates early**—saving time on unnecessary passes.

✅ Optimized Bubble Sort in C++ (With Early Exit Flag)

cpp

```cpp
#include <iostream>
using namespace std;

void optimizedBubbleSort(int array[], int n) {
    bool swapped;

    for (int i = 0; i < n - 1; ++i) {
        swapped = false;  // No swaps yet for this pass

        for (int j = 0; j < n - i - 1; ++j) {
            if (array[j] > array[j + 1]) {
                // Swap adjacent elements
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;

                swapped = true;  // A swap occurred
            }
        }

        // If no two elements were swapped, the array is already sorted
        if (!swapped) {
            break;
        }
    }
}

int main() {
    int data[] = {64, 25, 12, 22, 11};
    int n = sizeof(data) / sizeof(data[0]);

    optimizedBubbleSort(data, n);

    cout << "Sorted array: ";
```

```
    for (int i = 0; i < n; ++i)
        cout << data[i] << " ";

    return 0;
}
```

🧠 Optimized Bubble Sort Pseudo Code

pgsql

```
procedure OptimizedBubbleSort(array A, integer n)
    for i from 0 to n - 2 do
        swapped ← false

        for j from 0 to n - i - 2 do
            if A[j] > A[j + 1] then
                swap A[j] and A[j + 1]
                swapped ← true
            end if
        end for

        if swapped = false then
            break   // No swaps: array is already sorted
        end if
    end for
end procedure
```

🔍 What is Improved and How?

Feature	Standard Bubble Sort	Optimized Bubble Sort
All Passes Executed?	Yes (even if already sorted)	No (exits early if no swap is made)
Time Complexity (Best)	$O(n^2)$ even if sorted	O(n) when array is already sorted

Feature	Standard Bubble Sort	Optimized Bubble Sort
Use of Flag	No	Yes (`swapped` tracks if any change occurs)
Unnecessary Loops Avoided?	❌	✅

✅ Example: Already Sorted Array

Input:

`[1, 2, 3, 4, 5]`

- **Standard Version:** Still goes through all `n-1` passes.
- **Optimized Version:** Completes after **1 pass** because no swaps are made — detects sorted state!

Would you like to see a diagram or trace of how it works on an example?

give me an example and show step by step how flagged version works better

Absolutely! Let's walk through a **step-by-step example** using the **optimized bubble sort (flagged version)** to show **why and how** it performs better.

🔢 Example Array:

```cpp
int A[] = {1, 2, 3, 4, 5};
```

This array is **already sorted**.

⚙️ Optimized Bubble Sort Execution Steps

We'll show:

- Each **pass** through the array,
- Each **comparison and swap** attempt,
- Status of the `swapped` **flag**,
- Decision to **continue or stop**.

🔁 Pass 1 (i = 0)

- `swapped = false`

j	A[j]	A[j+1]	Compare	Action	swapped
0	1	2	1 > 2?	No Swap	false
1	2	3	2 > 3?	No Swap	false
2	3	4	3 > 4?	No Swap	false
3	4	5	4 > 5?	No Swap	false

➡️ At the end of **Pass 1**, `swapped = false` ➡️ **Array is already sorted**
✅ **Loop breaks early** after **just one pass**!

🔁 What Happens in Standard Bubble Sort?

Even though the array is sorted, **all n - 1 passes** will run.

For the same array `{1, 2, 3, 4, 5}`, the standard version will still do:

- Pass 1: 4 comparisons
- Pass 2: 3 comparisons
- Pass 3: 2 comparisons
- Pass 4: 1 comparison

🔁 **Total comparisons = 10**

But in **optimized version**, we only needed **4 comparisons in 1 pass**!

✅ Why Flagged Version is Better

Scenario	Standard Bubble Sort	Optimized Version (Flagged)
Already Sorted	Still does all passes	Exits after 1 pass
Partially Sorted	May still run full	Stops once sorted
Worst Case (Reversed)	Same runtime	Slight improvement possible

---

📌 Conclusion

The **flagged version saves time** by recognizing sorted arrays early, especially:

- For nearly sorted arrays: significant performance boost
- For sorted arrays: reduces time from **$O(n^2)$** to **$O(n)$**

Would you like to see this with a **partially sorted array** example too?

Show me range limitting bubble sort, explain the changes in details and an example describing how it works

Great! Let's dive into the **Range-Limiting Bubble Sort**, another optimization over the basic Bubble Sort algorithm.

🧠 What is **Range-Limiting Bubble Sort?**

In standard Bubble Sort, every pass compares the entire unsorted portion of the array—even if parts are already sorted.

Range-limiting Bubble Sort optimizes this by tracking the **position of the last swap**. Any

elements **after that last swap** are already in order, so there's **no need to compare them** again in the next pass.

✅ Key Improvement

- Track the **last index** where a swap occurred in a pass.
- In the next pass, only iterate **up to that index**.
- This limits the range of comparisons as the sorted portion grows from the back.

🔧 Code for Range-Limiting Bubble Sort (C++)

cpp

```cpp
#include <iostream>
using namespace std;

void rangeLimitingBubbleSort(int A[], int n) {
    int newN;
    while (n > 1) {
        newN = 0;  // Position of last swap
        for (int i = 1; i < n; ++i) {
            if (A[i - 1] > A[i]) {
                // Swap elements
                int temp = A[i - 1];
                A[i - 1] = A[i];
                A[i] = temp;

                newN = i;  // Update last swap index
            }
        }
        n = newN;  // Next pass only needs to go up to newN
    }
}

int main() {
    int arr[] = {5, 1, 4, 2, 8};
```

```
    int size = sizeof(arr) / sizeof(arr[0]);

    rangeLimitingBubbleSort(arr, size);

    cout << "Sorted array: ";
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    return 0;
}
```

🗒️ Pseudo Code

```
css

procedure RangeLimitingBubbleSort(array A, integer n)
    repeat
        newN ← 0
        for i from 1 to n - 1 do
            if A[i - 1] > A[i] then
                swap A[i - 1] and A[i]
                newN ← i
            end if
        end for
        n ← newN
    until n ≤ 1
end procedure
```

🔍 Example: `{5, 1, 4, 2, 8}`

We'll walk through **Pass by Pass** with **range limiting**.

🔁 Pass 1 (n = 5)

Compare and swap:

- (5,1) → swap → `{1,5,4,2,8}` → last swap at index **1**

- (5,4) → swap → `{1,4,5,2,8}` → last swap at **2**
- (5,2) → swap → `{1,4,2,5,8}` → last swap at **3**
- (5,8) → no swap

🔸 Last swap index = **3**, so next time only go up to `n = 3`.

🔁 Pass 2 (n = 3)

- (1,4) → no swap
- (4,2) → swap → `{1,2,4,5,8}` → last swap at **2**

🔸 Next n = 2

🔁 Pass 3 (n = 2)

- (1,2) → no swap → last swap index = **0**

🔚 Done! Array is sorted: `{1,2,4,5,8}`

✅ How It's Better

Feature	Basic Bubble Sort	Range-Limiting Version
Always goes full range?	Yes	No
Uses last swap to limit?	❌	✅
Comparisons after sorted part	Still performed	Skipped
Best-case Time Complexity	$O(n^2)$	O(n) (if already sorted early)

🚀 When It Helps Most

- **Partially sorted arrays**: avoids needless comparisons.
- **Worst-case** (fully reversed): still $O(n^2)$, but might slightly help.

- **Best-case** (already sorted): still optimizes to O(n), like flagged version.

Would you like a **side-by-side comparison table** of all three versions?

Show me range alternating bubble sort, explain the changes in details and an example describing how it works

Absolutely! Let's now explore the **Alternating Bubble Sort** — also known as the **Cocktail Shaker Sort** or **Bidirectional Bubble Sort** — which is another improvement over basic bubble sort.

🧠 What is **Alternating Bubble Sort?**

Idea:
Instead of only "bubbling up" the largest element from left to right (like regular bubble sort), alternating bubble sort **also bubbles the smallest element from right to left** in the same pass.

🔄 This means:

- One pass = a **forward pass** (left → right) **and** a **backward pass** (right → left).
- So we're **shrinking the unsorted range from both ends** each time — more efficient!

✅ Key Improvements Over Basic Bubble Sort

Feature	Basic Bubble Sort	Alternating Bubble Sort
Direction of sorting	Left to right only	Alternates: left→right then right→left
Sorted range shrinkage	Only from end	From both ends
Swap detection (flag)?	Yes (can include it)	Yes (recommended)

Feature	Basic Bubble Sort	Alternating Bubble Sort
Best-case performance	O(n) with flag	O(n) with flag
Useful when	Data is almost sorted or with swapped min/max	

🔧 C++ Code: Alternating Bubble Sort

cpp

```cpp
#include <iostream>
using namespace std;

void alternatingBubbleSort(int A[], int n) {
    bool swapped = true;
    int start = 0;
    int end = n - 1;

    while (swapped) {
        swapped = false;

        // Forward pass: bubble the largest to the right
        for (int i = start; i < end; ++i) {
            if (A[i] > A[i + 1]) {
                swap(A[i], A[i + 1]);
                swapped = true;
            }
        }

        // If no swaps were made, array is sorted
        if (!swapped)
            break;

        // Decrease end as the largest is now sorted
        --end;

        swapped = false;
```

```
        // Backward pass: bubble the smallest to the left
        for (int i = end; i > start; --i) {
            if (A[i - 1] > A[i]) {
                swap(A[i - 1], A[i]);
                swapped = true;
            }
        }

        // Increase start as the smallest is now sorted
        ++start;
    }
}

int main() {
    int arr[] = {3, 1, 4, 5, 2};
    int n = sizeof(arr) / sizeof(arr[0]);

    alternatingBubbleSort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
    return 0;
}
```

🗒️ Pseudo Code

pgsql

```
procedure AlternatingBubbleSort(array A, integer n)
    start ← 0
    end ← n - 1
    swapped ← true

    while swapped do
        swapped ← false

        // Forward pass
```

```
        for i from start to end - 1 do
            if A[i] > A[i + 1] then
                swap A[i], A[i + 1]
                swapped ← true
            end if
        end for

        if not swapped then
            break

        end ← end - 1
        swapped ← false

        // Backward pass
        for i from end to start + 1 do
            if A[i - 1] > A[i] then
                swap A[i - 1], A[i]
                swapped ← true
            end if
        end for

        start ← start + 1
    end while
end procedure
```

🧪 Example Walkthrough: `{3, 1, 4, 5, 2}`

🔁 Pass 1:

- Forward:
  `{3,1,4,5,2}` → `{1,3,4,5,2}` → `{1,3,4,2,5}`
  Largest (5) is at end ✅
- Backward:
  `{1,3,4,2,5}` → `{1,3,2,4,5}` → `{1,2,3,4,5}`
  Smallest (1) is at start ✅

🔁 Pass 2:

- Forward:
  No swap (array already sorted) → stop early

✅ Benefits Summary

Benefit	Description
Two-way sorting	Finds both min and max each pass
Efficient for "swapped min/max"	Good when both smallest and largest are misplaced
Early stopping with flag	Stops if no swaps happen
Performance (Best-case)	O(n)
Performance (Average/Worst)	$O(n^2)$, but usually better than basic bubble sort

---

Would you like a **visual animation** or **comparison table of all optimizations** next?